Uncertainty Estimation in Regression Using Additive Models and Gaussian Processes

A THESIS

submitted in partial fulfillment of the requirements
for the award of the dual degree of

Bachelor of Science - Master of Science

in

Data Science and Engineering

by

Amish Anand
(20036)

Department of Data Science and Engineering
INDIAN INSTITUTE OF SCIENCE EDUCATION AND RESEARCH BHOPAL
BHOPAL - 462066
April 2025

भारतीय विज्ञान शिक्षा एवं अनुसंधान संस्थान, भोपाल
Indian Institute of Science Education and Research, Bhopal
Department of Data Science and Engineering
Bhopal Bypass Road, Bhauri Bhopal 462 066

CERTIFICATE

This is to certify that **Amish Anand**, BS-MS (Data Science and Engineering), has worked on the project entitled **' Uncertainty Estimation in Regression Using Additive Models and Gaussian Processes'** under my supervision and guidance. The content of this report is original and has not been submitted elsewhere for the award of any academic or professional degree.

Dr Vinod Kurmi (PI)

April 2025
IISER Bhopal

Committee Member	Signature	Date
Dr. Vaibhav Kumar	_____	_____
Dr. Jasabanta Patro	_____	_____
Dr. Samiran Das	_____	_____

ACADEMIC INTEGRITY AND COPYRIGHT DISCLAIMER

I hereby declare that this project is my own work and, to the best of my knowledge, it contains no materials previously published or written by another person, or substantial proportions of material which have been accepted for the award of any other degree or diploma at IISER Bhopal or any other educational institution, except where due acknowledgement is made in the document.

I certify that all copyrighted material incorporated into this document is in compliance with the Indian Copyright Act (1957) and that I have received written permission from the copyright owners for my use of their work, which is beyond the scope of the law. I agree to indemnify and save harmless IISER Bhopal from any and all claims that may be asserted or that may arise from any copyright violation.

April 2025 **Amish Anand**
IISER Bhopal

ACKNOWLEDGMENT

I sincerely thank my parents and my younger brother, Ankit, for their support, love, and understanding. Whenever I felt hopeless they always stood by me with their belief. Their presence has been a pillar of strength, and I am deeply grateful for everything they have done for me. I would like to express my heartfelt gratitude to my late Dadima and Dadaji. Although they are no longer with me, I have always felt their presence. I also want to thank Nani Ma, Nana Ji, Mausi Ma and mamaji for their constant love and support .

I am deeply grateful to my PI, Dr.Vinod Kurmi, for his invaluable guidance ,mentorship and his support during the tough times . His feedback have shaped not only my research but also my academic and personal growth.

I would like to thank all my friends and seniors Shambhu, Naksh, Sayantan, Arpita, Vaibhav, Rita di and many more who insisted not to take the BS exit. I am thankfull to all my iiserb and non iiserb friends for their support .I am also thankfull to all the VISDOM lab memebers especially Udbhav Bhaiya, Rajeev Bhaiya and Rini Mam for their support.

Finally, I extend my gratitude to all the DSE faculties of IISER Bhopal . Their collective efforts and dedication have played an important role in helping me navigate this academic path.I also want to thank Dr.Biswajit Patra for being the Co-PI in my BS thesis

Amish Anand

ABSTRACT

This study introduces a hybrid model integrating Random Fourier Features (RFF) ,Gaussian Process and Neural Additive Models for uncertainty estimation in regression tasks. Gaussian Process Neural Additive Models (GP-NAM) leverages a neural additive structure, allowing each feature to contribute independently to the target prediction, enhancing interpretability. By applying RFF, we efficiently approximate complex, non-linear interactions in a computationally feasible manner. Uncertainty quantification is achieved through a sequential application of three Gaussian Processes (GPs): the first models residuals to enhance prediction accuracy, the second estimates heteroscedastic variance to capture spatially varying uncertainty, and the third further refines residuals for improved prediction. The proposed model demonstrates good performance across multiple datasets, evaluated using Root Mean Square Error (RMSE), Expected Calibration Error (ECE), and Negative Log Predictive Density (NLPD).

LIST OF SYMBOLS OR ABBREVIATIONS

Random Fourier Features	RFF
Gaussian Processes	GPs
Neural Additive Models	NAM
Gaussian Mixture Models	GMM
Mixture Density Network	MDN
Root Mean Square Error	RMSE
Expected Calibration Error	ECE
Negative Log Predictive Density	NLPD
Bayesian neural network	BNN
Gaussian Process Neural Additive Models	GP-NAM

LIST OF FIGURES

3.1 Representation of GP prediction 10
3.2 Representation of Gaussian Mixture Model 20
3.3 Representation of the Mixture Density Network model 22

LIST OF TABLES

4.1 Performance comparison (RMSE ↓, ECE ↓, and NLPD ↓) across multiple datasets. Minimum values for each metric are highlighted in bold. 34
4.2 Evaluation results of Bayesian Neural Network on different datasets 36
4.3 Evaluation results of Bayesian Linear Regression on different datasets 36
4.4 Evaluation results of quantile-regression on different datasets. . . . 36
4.5 Evaluation results of Natural Gradient Boosting on different datasets. 36
4.6 Evaluation results of Gaussian Process Regressor on different datasets. 37

Contents

Certificate **i**

Academic Integrity and Copyright Disclaimer **ii**

Acknowledgment **iii**

Abstract **iv**

List of Symbols or Abbreviations **v**

List of Figures **vi**

List of Tables **vii**

1 Introduction **1**

1.1 Objective 3

2 Literature Review **4**

2.1 Related Work 4

2.2 Research Gaps 6

3 Methodology **8**

3.1 Gaussian Processes 8

3.2 PROPOSED METHOD 11

3.3 Framework with Gaussian Mixture Model 18

3.4 Framework with Neural Network-based Mixture Density Network . 20

3.5 Analysis of Gaussian Process Optimal Configurations 24
3.5.1 Objective and White Noise Criteria 24
3.5.2 Testing Procedures 25
3.5.3 Results 26
3.5.4 Limitations 27
3.5.5 Conclusion 27
3.6 Interpretability Measures and Their Drawbacks 28
3.6.1 Incorporation of Interpretability Measures 28
3.6.2 Interpretability Index 30
3.6.3 Drawbacks and Limitations 30
3.6.4 Results 31
3.6.5 Conclusion 32

4 Results and Discussion **33**
4.1 Evaluation Metrics 33
4.2 Results 34
4.3 Results Comparison 37
4.4 Conclusion 38
4.5 Future Works 38

5 Appendix **39**
5.1 Variational Sparse Gaussian Process Approximation 39

Bibliography **40**

Chapter 1

Introduction

Regression analysis is used extensively to make continuous prediction in a wide range of applications including economics, engineering, medicine, and environmental science. Standard regression techniques emphasize providing point predictions—single value estimates of the target variable. Yet, in most real-world applications, such estimates cannot be used in isolation without a supporting estimate of uncertainty. Estimation of uncertainty in regression fills this essential gap by quantifying the variability or confidence of the prediction and thus offering a wider understanding of the outputs of the model. This ability becomes critical in situations where decisions have serious implications, e.g, in medical diagnosis, financial forecasting, robot navigation, and climate modeling. Beyond the importance of uncertainty in risk evaluation, there are areas of safety-related applications where knowledge of uncertainty can avoid devastating failures by alerting to unreliable prediction. Moreover, uncertainty quantification gives users a way to be transparent and gives stakeholders a better understanding of when and why a prediction could not work in complex models like deep neural networks, where a lack of clear understanding of the model becomes a significant problem.

Recent research in uncertainty estimation has been inspired by advances in statistical and machine learning methodologies. Classical statistical methods such as the Gaussian process and the Bayesian linear regression offer time-honored means of uncertainty quantification but lack scalability and often inflexibility in high-

dimensional applications. Conversely, modern approaches such as the Bayesian neural network, ensemble techniques, and conformal prediction utilize computing advancements to overcome such inadequacies and allow uncertainty quantification in high-dimensional, non-linear, and high-dimensional applications. In addition, the incorporation of uncertainty-aware techniques into real-world applications has encouraged advances in active learning, robust optimization, and decision making under uncertainty. An example is the often employed Kalman filter tracking algorithm , which necessitates the variance of the observed object's location estimation to accurately integrate past and present information, hence enhancing the estimation of the current state of the tracked object [1]

Even with these improvements, challenges remain. Correct differentation of the aleatoric and epistemic uncertainties and the maintenance of computational efficiency are ongoing areas of research. In addition, the usability and interpretability of uncertainty estimates for end-users, especially in cross-disciplinary applications, must be carefully addressed to span the gap between theoretical innovation and real-world deployment.

Evaluation of uncertainty estimates also presents unique challenges. Metrics such as coverage probability, calibration error, and sharpness provide quantitative measures of uncertainty quality, but their interpretation can vary across applications[2]. For instance, in medical diagnostics, overly conservative uncertainty estimates may lead to excessive caution, while overly optimistic estimates may result in undue confidence. Developing standardized evaluation frameworks that balance these trade-offs is an active area of research. Additionally, the rise of large-scale, heterogeneous datasets and the increasing complexity of modern models necessitate scalable and robust uncertainty estimation methods that can generalize across diverse settings.

This study addresses these challenges by proposing a hybrid approach that integrates Random Fourier Features [3](RFF), Gaussian Processes [4](GPs), and Neural Additive Models (NAMs)[5] to achieve uncertainty estimation in regression tasks. Neural Additive Models are particularly well-suited for interpretable machine learning as they independently represent each feature's contribution to the

prediction which enhances transparency in the model's decision-making process. By employing RFF, we efficiently approximate complex, non-linear relationships in a computationally feasible manner, making the approach suitable for large-scale datasets. Additionally, the inclusion of Gaussian Processes enables uncertainty quantification, which is critical for applications requiring an understanding of prediction confidence.

The proposed method uses Gaussian Process Neural Additive Models (GP-NAM) for obtaining accurate point predictions. For point predictions, the model combines NAMs with RFF to construct Gaussian processes for each feature, leveraging a single-layer neural network architecture that maintains interpretability and reduces computational complexity. Following this, uncertainty quantification is achieved through a sequential application of three Gaussian Processes: the first GP models the residuals to capture feature-target relationships not addressed in the initial prediction, the second GP models heteroscedastic variance to capture spatially varying uncertainty, and the third GP refines residuals to enhance overall prediction precision.

This hybrid GP-NAM model was evaluated on multiple datasets using performance metrics including Root Mean Square Error (RMSE), Expected Calibration Error (ECE), and Negative Log Predictive Density (NLPD).

1.1 Objective

The objective is to develop a hybrid model that integrates Random Fourier Features (RFF), Gaussian Processes (GPs), and Neural Additive Models (NAMs) to enable accurate uncertainty estimation in regression tasks. Specifically, the proposed model aims to leverage Random Fourier Features (RFF) to efficiently approximate complex, non-linear interactions within the data and to integrate Gaussian Processes (GPs). The model uses a sequential application of three GPs: The first GP models the residuals to improve prediction accuracy,the second GP captures heteroscedastic variance, accounting for spatially varying uncertainty across the data,the third GP further refines the residuals to provide an additional layer of uncertainty quantification for improved predictions.

Chapter 2

Literature Review

2.1 Related Work

The estimation of uncertainty in regression tasks has emerged as a critical research area, where models are increasingly deployed in high-stakes applications where understanding predictive reliability is essential. The literature on regression uncertainty estimation is divided into two main sources of uncertainty: aleatoric and epistemic [6]. Aleatoric uncertainty captures inherent noise in the data that cannot be reduced even with more data, whereas epistemic uncertainty arises from limited knowledge about the model parameters and can potentially be reduced by acquiring additional data or refining the model structure.

Gaussian Processes (GPs) have long been utilized for uncertainty estimation due to their ability to produce probabilistic outputs. GPs model the distribution of potential outputs based on observed data, thereby providing a measure of uncertainty along with each prediction [7]. GPs provide a structured framework for uncertainty quantification, but their computational complexity grows with the size of the dataset, making them impractical for large-scale applications. To address this limitation, researchers have developed scalable approximations to GPs, such as the use of inducing points or sparse approximations [8].The introduction of Random Fourier Features (RFF) by [3] provided a breakthrough in this area, allowing for computationally efficient approximations of kernel functions by mapping data

into a randomized low-dimensional feature space.

Hybrid models that integrate neural networks with Gaussian Processes have been developed to harness the strengths of both approaches, combining the scalability and flexibility of deep learning with the probabilistic nature of GPs. Gaussian Process Neural Additive Models (GP-NAM) [9] represent an innovative approach in this space, leveraging a single-layer neural network to construct Gaussian processes for each feature using RFF. This method allows for both interpretability and efficient large-scale training.

Uncertainty estimation in neural networks has received substantial attention because of its critical role in high-stakes fields such as autonomous driving, healthcare, and finance, where understanding the confidence of model predictions is essential for safety and reliability [10]. Bayesian neural networks have emerged as a popular approach for uncertainty estimation but they can be challenging to train and often require complex variational inference techniques [11] . BNNs offer a principled way to model uncertainty but their computational demands are often high. An influential contribution by Gal and Ghahramani [12] demonstrated that applying dropout at test time can serve as a form of approximate Bayesian inference. This Monte Carlo dropout technique efficiently estimates epistemic uncertainty in standard deep neural networks without requiring extensive modifications to the model architecture.

Ensemble methods, where multiple neural networks are trained to produce a distribution of predictions, have also been explored for uncertainty estimation. These methods improve robustness and provide a sense of prediction confidence but they typically demand significant computational resources [13].

Several methods have been proposed to estimate uncertainty directly by predicting intervals. Quantile regression[14], for example, estimates specific quantiles of the response distribution by constructing prediction intervals that mirror the variability of the data. Conformal prediction[15] techniques also offer a way to generate prediction intervals with formal statistical guarantees under minimal assumptions.

Recent research has focused on integrating multiple sources of uncertainty into unified models. [16] propose a framework that jointly models aleatoric and epis-

temic uncertainties within deep learning systems. Such hybrid approaches have proven effective in domains like computer vision and autonomous systems, where both data noise and model uncertainty critically impact decision-making.

The proposed approach builds upon these developments by combining RFF, GPs, and NAMs [5]to provide a hybrid model that achieves both high predictive accuracy and interpretability in uncertainty estimation. Unlike standard GPs, which are computationally prohibitive on large datasets, the use of RFF allows for scalable approximations of Gaussian processes. Moreover, by employing a sequential application of three GPs, the model captures residuals, heteroscedastic variance, and any remaining errors with high granularity. This layered approach enhances the model's ability to provide reliable uncertainty estimates, offering a structured and interpretable method for analyzing complex, non-linear relationships. The integration of NAMs further ensures that each feature's individual contribution remains transparent, addressing the interpretability concerns often associated with deep learning models.

2.2 Research Gaps

Recent advancements in regression models have significantly enhanced predictive capabilities but they often suffer from poor scalability and a lack of interpretability. Traditional regression techniques which are interpretable struggle with high-dimensional and complex data. However, more flexible models such as deep neural networks lack transparency, making it difficult for users to understand and trust their predictions. Current solutions like Bayesian neural networks offer probabilistic interpretations and uncertainty estimates, but they are computationally intensive. Computational costs increases with larger datasets even with approximations like variational inference or Monte Carlo dropout[17]. Similarly, Gaussian processes , known for their excellent uncertainty quantification, become computationally unfeasible as the dataset size increases[18].

The evaluation and calibration of uncertainty predictions is a critical area with significant gaps. Current metrics, such as those based on credible intervals, often average over the dataset, potentially masking overconfident predictions by failing

to ensure pointwise coverage. For example, studies have shown that random uncertainty estimates can appear well-calibrated under existing definitions, which is problematic for safety-critical applications. New calibration approaches, such as the Expected Normalized Calibration Error , have been proposed, but they require broader validation across domains to ensure robustness[19] . The development of metrics that prioritize pointwise coverage and handle complex data distributions remains a key research need.

Accurately separating aleatoric and epistemic uncertainty remains challenging. Many methods, such as Monte Carlo Dropout or ensemble-based approaches, struggle to clearly separate these components, leading to misleading uncertainty estimates. This is problematic in applications like outlier detection, where understanding the source of uncertainty is crucial[20].

Therefore,there is a significant gap in integrating methods that can efficiently handle the complexity and size of modern datasets while maintaining model transparency and reliability.

Chapter 3

Methodology

3.1 Gaussian Processes

Overview

A *Gaussian Process (GP)* is a non-parametric, Bayesian approach used for regression and classification. A GP describes a distribution over functions instead of parametric models, which presume a fixed shape for the function.This flexibility allows it to adapt to complex data structures while naturally quantifying uncertainty in its predictions.[7]

Mathematical Formulation

A Gaussian Process is fully specified by its mean function $m(x)$ and covariance function $k(x, x')$. Formally, we write:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

where:

- $m(x) = \mathbb{E}[f(x)]$ represents the expected value of the function at input x.
- $k(x, x') = \mathbb{E}\left[(f(x) - m(x))(f(x') - m(x'))\right]$ defines the covariance between function values at inputs x and x'.

The Covariance Function (Kernel)

The covariance function (or kernel) is central to a GP. It encodes assumptions about the function's properties, such as smoothness and periodicity. Common choices include:

- **Squared Exponential (RBF) Kernel:**

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

  where σ_f^2 is the signal variance and ℓ is the length scale parameter.

- **Matern Kernel:** Offers flexibility by adjusting the smoothness of the resulting functions through an additional parameter.

- **Periodic Kernel:** Useful when the underlying function exhibits periodic behavior.

Inference with Gaussian Processes

Given a set of training data $\mathbf{X} = \{x_1, x_2, \ldots, x_n\}$ with corresponding outputs $\mathbf{y}$, the goal is to predict the function value f_* at a new input x_*. Under the GP framework, the joint distribution of the observed outputs and the function value at x_* is:

$$\begin{pmatrix} \mathbf{y} \\ f_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} m(\mathbf{X}) \\ m(x_*) \end{pmatrix}, \begin{pmatrix} K(\mathbf{X}, \mathbf{X}) + \sigma_n^2 I & K(\mathbf{X}, x_*) \\ K(x_*, \mathbf{X}) & K(x_*, x_*) \end{pmatrix}\right)$$

Here, $K(\mathbf{X}, \mathbf{X})$ is the covariance matrix computed over the training inputs, $K(\mathbf{X}, x_*)$ is the covariance between training inputs and the new input, and σ_n^2 represents the noise variance.

The predictive distribution for f_* is then obtained by conditioning on the observed data, resulting in:

- **Predictive Mean:**

$$\bar{f}_* = m(x_*) + K(x_*, \mathbf{X}) \left[K(\mathbf{X}, \mathbf{X}) + \sigma_n^2 I\right]^{-1} (\mathbf{y} - m(\mathbf{X}))$$

- **Predictive Variance:**

$$\mathrm{var}(f_*) = K(x_*, x_*) - K(x_*, \mathbf{X}) \left[K(\mathbf{X}, \mathbf{X}) + \sigma_n^2 I\right]^{-1} K(\mathbf{X}, x_*)$$

Advantages and Limitations

Advantages:

- **Flexibility:** Being non-parametric, GPs can model complex functions without assuming a specific functional form.
- **Uncertainty Quantification:** They provide a natural measure of uncertainty in the predictions, which is crucial for many applications.
- **Bayesian Inference:** The GP framework allows for the incorporation of prior knowledge and updates it with observed data.

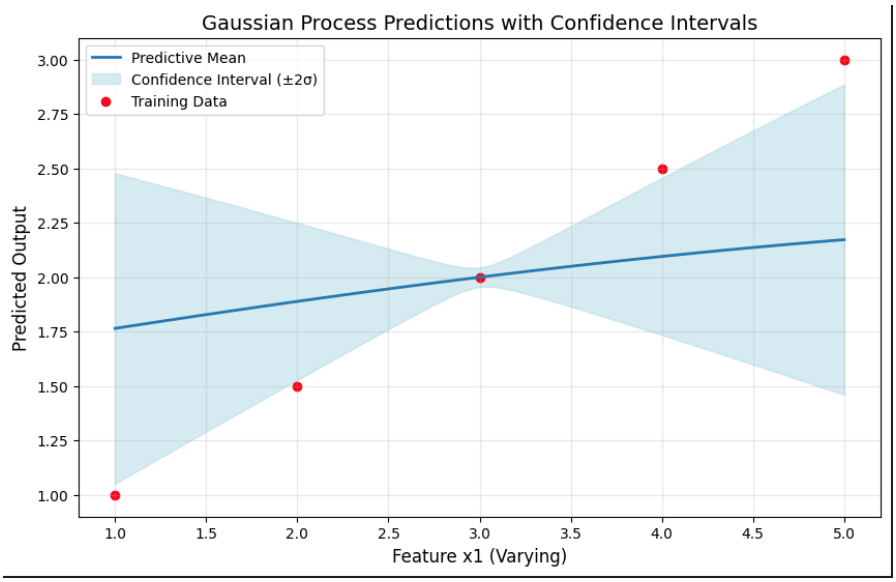

Figure 3.1: Representation of GP prediction

Limitations:

- **Computational Cost:** GP inference involves operations that scale cubically with the number of training points, which can be computationally intensive for large datasets.

- **Kernel Selection:** The performance of a GP model depends heavily on the choice and parameterization of the kernel function.

Applications

Gaussian Processes are widely used in fields such as machine learning, statistics, and time-series analysis. Their ability to provide probabilistic predictions makes them particularly useful in scenarios where quantifying uncertainty is as important as making accurate predictions.

3.2 PROPOSED METHOD

Our proposed methodology integrates three main components—Random Fourier Features (RFF), Gaussian Processes (GPs), and Neural Additive Models (NAMs)—to develop a hybrid model for uncertainty estimation. This section details each component, their integration, and the sequential approach used to achieve both point predictions and uncertainty quantification.

Building on the interpretability of NAMs and the scalability provided by RFF, our methodology uses Gaussian Process Neural Additive Models (GP-NAM) [9]to estimate both point predictions and uncertainties . NAMs restrict the architecture of neural networks to yield a family of interpretable "glass-box" models. These models learn a linear combination of networks, each attending to a single input feature. This approach allows for complex, feature-specific shape functions that are trained jointly, enhancing interpretability without sacrificing the power of deep learning.

GP-NAM leverages a single-layer neural network to construct a Gaussian process for each feature by employing Random Fourier Features (RFF). This method introduces a way to accelerate kernel machine training by mapping input data into a randomized low-dimensional feature space using random Fourier features, which enables the use of faster linear algorithms to approximate kernel methods, providing a scalable alternative to traditional, computationally intensive kernel approaches. This approximation allows the model to capture complex, non-linear relationships in a computationally efficient manner.

Key steps in this approach include:

- Transforming the feature space using a finite set of cosine functions.
- Reducing model complexity by employing a single neural network layer with shared parameters across features, making the model both interpretable and computationally efficient.

GP-NAM simplifies the model architecture while preserving accuracy and interpretability by assigning each feature its own contribution to the target variable, which can be individually inspected. This makes GP-NAM particularly valuable in applications that require transparency in model decision-making.

Once the model has generated a point prediction we need to understand how confident the model is in that prediction. To achieve this, we use a sequential application of three Gaussian Processes (GPs), each targeting a specific aspect of the residuals.

1. The first GP models the mean residuals, effectively learning additional relationships between features and the target that were not captured. This GP uses a Squared Exponential kernel, which provides a smooth flexible way to model the relationships within the residuals. Thus, the accuracy of the prediction is significantly improved.
2. The second GP *HetGP* [21]models the heteroscedastic variance, which is the property of having non-uniform variance across the data. This GP uses a heteroscedastic likelihood function, which allows it to learn the uncertainty associated with each prediction. The kernel used here is a combination of Squared Exponential kernels for different input features, enabling the GP to model the changing variance of the residuals across the input dimensions. It allows the model to learn where the predictions tend to be less or more uncertain.
3. The third GP addresses any remaining residual errors after the first GP has modeled the residuals. It is trained on the remaining residuals, which are essentially the difference between the original residuals from the model and the contribution made by the first GP. The third GP, also using a Squared

Exponential kernel, further refines the model predictions.

The final prediction of the target variable is obtained by combining the contributions from the different components:

- Base Prediction from the RFF model.
- Mean Residual Correction from the first GP.
- Additional Residual Patterns from the third GP.

The HetGP, which models heteroscedastic uncertainty, provides a variance estimate that represents how confident the model is in its predictions. This combination allows the model to not only make accurate predictions but also provide a measure of reliability for each prediction.

To overcome the computational complexity and the intractable posterior for GPs many approximate inference methods are proposed including MCMC[21], Variational Bayes[22], Laplace Approximation[23], Expectation Propogation[24], here we adopt the variational sparse approximation of GP through inducing points [25], which speeds up the computation by choosing inducing points over the covariate space.

Joint Training of the GP-NAM Model with Residual Modeling

Base Model (GP-NAM)

Given an input $x \in \mathbb{R}^d$, the RFF mapping is defined as:

$$\phi(x) = \begin{bmatrix} \sqrt{\frac{2}{S}} \cos\left(\frac{z_1 x_1}{b_1} + c_1\right) \\ \vdots \\ \sqrt{\frac{2}{S}} \cos\left(\frac{z_S x_1}{b_1} + c_S\right) \\ \vdots \\ \sqrt{\frac{2}{S}} \cos\left(\frac{z_1 x_d}{b_d} + c_1\right) \\ \vdots \\ \sqrt{\frac{2}{S}} \cos\left(\frac{z_S x_d}{b_d} + c_S\right) \end{bmatrix},$$

where:

- S is the number of random features per input dimension,
- $z_s \sim \mathcal{N}(0, 1)$ are random frequencies,
- $c_s \sim \text{Uniform}(0, 2\pi)$ are phase shifts,
- b_i are feature-specific kernel width parameters.

The base model then predicts:

$$\hat{y}_{\text{base}}(x) = \phi(x)^\top w,$$

where $w \in \mathbb{R}^D$ (with $D = 1 + dS$) is the trainable weight vector. The loss function for the base model is given by a regularized squared error:

$$L_{\text{base}}(w) = \frac{1}{2} \sum_{n=1}^{N} \Big(y_n - \phi(x_n)^\top w \Big)^2 + \frac{1}{2} \|w\|^2,$$

with y_n being the target for the n-th sample and N the number of training samples. The residuals are then defined as:

$$r_n = y_n - \hat{y}_{\text{base}}(x_n).$$

GP Modules for Residual Modeling

To correct the residuals and quantify uncertainty, we introduce three GP modules.

GP1: Mean Residual Correction GP1 is used to model the systematic component of the residuals. We assume a Gaussian process prior:

$$f_1(x) \sim \mathrm{GP}\big(0, k_1(x, x')\big),$$

with kernel $k_1(x, x')$ (e.g., Squared Exponential). With inducing points Z_1 and variational distribution $q(u_1) = \mathcal{N}(m_1, S_1)$, where $u_1 = f_1(Z_1)$, the predictive distribution for a new input x is approximated by:

$$q\big(f_1(x)\big) = \mathcal{N}\big(\mu_1(x), \sigma_1^2(x)\big),$$

with:

$$\mu_1(x) = k_1(x, Z_1) K_{Z_1 Z_1}^{-1} m_1,$$
$$\sigma_1^2(x) = k_1(x, x) - k_1(x, Z_1) K_{Z_1 Z_1}^{-1} \big(K_{Z_1 Z_1} - S_1\big) K_{Z_1 Z_1}^{-1} k_1(Z_1, x).$$

The negative Evidence Lower Bound (ELBO) for GP1 is then:

$$L_{\mathrm{GP1}} = -\sum_{n=1}^{N} \mathbb{E}_{q(f_1(x_n))}\Big[\log \mathcal{N}\big(r_n \mid f_1(x_n), \sigma_1^2\big)\Big] + \mathrm{KL}\big(q(u_1) \,\|\, p(u_1)\big).$$

GP2: Heteroscedastic Uncertainty Estimation GP2 is designed to capture input-dependent noise. Its likelihood is:

$$p\big(r_n \mid f_2(x_n)\big) = \mathcal{N}\big(r_n \mid f_2(x_n), \sigma_2^2(x_n)\big),$$

with $\sigma_2^2(x_n)$ varying with x. Using inducing points Z_2 and variational distribution $q(u_2) = \mathcal{N}(m_2, S_2)$, the negative ELBO is given by:

$$L_{\text{GP2}} = -\sum_{n=1}^{N} \mathbb{E}_{q(f_2(x_n))}\Big[\log \mathcal{N}\big(r_n \mid f_2(x_n), \sigma_2^2(x_n)\big)\Big] + \text{KL}\big(q(u_2) \,\|\, p(u_2)\big).$$

GP3: Additional Residual Correction GP3 models any remaining error after GP1's correction. We define the new residual as:

$$r'_n = r_n - \mu_1(x_n).$$

Assuming a Gaussian likelihood with constant noise σ_3^2 and kernel $k_3(x, x')$, with inducing points Z_3 and variational distribution $q(u_3) = \mathcal{N}(m_3, S_3)$, the predictive distribution is:

$$q\big(f_3(x)\big) = \mathcal{N}\big(\mu_3(x), \sigma_3^2(x)\big).$$

The negative ELBO for GP3 is:

$$L_{\text{GP3}} = -\sum_{n=1}^{N} \mathbb{E}_{q(f_3(x_n))}\Big[\log \mathcal{N}\big(r'_n \mid f_3(x_n), \sigma_3^2\big)\Big] + \text{KL}\big(q(u_3) \,\|\, p(u_3)\big).$$

Total Joint Loss and Optimization

The overall training objective is the sum of the losses from the base model and all GP modules:

$$L_{\text{total}} = L_{\text{base}} + L_{\text{GP1}} + L_{\text{GP2}} + L_{\text{GP3}}.$$

Let θ_{base} denote the parameters of the base model (e.g., w and any trainable RFF parameters) and θ_i denote the parameters of GP_i (including variational parameters, kernel hyperparameters, etc.). The joint training problem is then:

$$\theta^* = \arg \min_{\theta = \{\theta_{\text{base}}, \theta_1, \theta_2, \theta_3\}} L_{\text{total}}(\theta).$$

Using the Adam optimizer with a learning rate η, the update step is:

$$\theta \leftarrow \theta - \eta \nabla_\theta L_{\text{total}}(\theta).$$

This single gradient update step simultaneously adjusts all trainable parameters, allowing the base model and GP modules to co-adapt in an end-to-end manner.

Summary

The joint training framework is mathematically represented as follows:

- **Base Model Loss**:

$$L_{\text{base}}(w) = \frac{1}{2} \sum_{n=1}^{N} \big(y_n - \phi(x_n)^\top w\big)^2 + \frac{1}{2} \|w\|^2. \tag{3.1}$$

- **GP1 Loss**:

$$L_{\text{GP1}} = -\sum_{n=1}^{N} \mathbb{E}_{q(f_1(x_n))} \Big[\log \mathcal{N}\big(r_n \mid f_1(x_n), \sigma_1^2\big)\Big] + \text{KL}\big(q(u_1) \,\|\, p(u_1)\big). \tag{3.2}$$

- **GP2 Loss**:

$$L_{\text{GP2}} = -\sum_{n=1}^{N} \mathbb{E}_{q(f_2(x_n))} \Big[\log \mathcal{N}\big(r_n \mid f_2(x_n), \sigma_2^2(x_n)\big)\Big] + \text{KL}\big(q(u_2) \,\|\, p(u_2)\big). \tag{3.3}$$

- **GP3 Loss**:

$$L_{\text{GP3}} = -\sum_{n=1}^{N} \mathbb{E}_{q(f_3(x_n))} \Big[\log \mathcal{N}\big(r'_n \mid f_3(x_n), \sigma_3^2\big)\Big] + \text{KL}\big(q(u_3) \,\|\, p(u_3)\big), \tag{3.4}$$

  where $r'_n = r_n - \mu_1(x_n)$.

- **Total Joint Loss**:

$$L_{\text{total}} = L_{\text{base}} + L_{\text{GP1}} + L_{\text{GP2}} + L_{\text{GP3}}. \tag{3.5}$$

- **Joint Optimization**:

$$\theta^* = \arg \min_{\theta=\{\theta_{\text{base}},\theta_1,\theta_2,\theta_3\}} L_{\text{total}}(\theta), \tag{3.6}$$

  with updates performed via

$$\theta \leftarrow \theta - \eta \nabla_\theta L_{\text{total}}(\theta), \tag{3.7}$$

  using the Adam optimizer.

3.3 Framework with Gaussian Mixture Model

Complementing the GP-NAM architecture, Gaussian Mixture Regression (GMR) [26][27][28]further refines predictions and quantifies residual uncertainty. After obtaining the initial prediction from GP-NAM,

the residual is computed as:

$$r(x) = y - \hat{y}_{\text{GPNAM}}(x).$$

We model the joint distribution of the inputs and residuals with a Gaussian mixture model:

$$p(x, r) = \sum_{k=1}^{K} \pi_k \mathcal{N}\left(\begin{pmatrix} x \\ r \end{pmatrix} \;\middle|\; \mu_k, \Sigma_k \right),$$

where each component k has:

- A mixing weight π_k (with $\sum_{k=1}^{K} \pi_k = 1$).
- A mean vector $\mu_k = \begin{pmatrix} \mu_k^x \\ \mu_k^r \end{pmatrix}$.

- A covariance matrix partitioned as

$$\Sigma_k = \begin{pmatrix} \Sigma_k^{xx} & \Sigma_k^{xr} \\ \Sigma_k^{rx} & \Sigma_k^{rr} \end{pmatrix}.$$

The number of components K is selected using the Bayesian Information Criterion (BIC). For a new input x^*, the conditional distribution of r is computed using Gaussian conditioning:

$$\mu_k^{r|x} = \mu_k^r + \Sigma_k^{rx}(\Sigma_k^{xx})^{-1}(x^* - \mu_k^x),$$

$$\sigma_k^{2|x} = \Sigma_k^{rr} - \Sigma_k^{rx}(\Sigma_k^{xx})^{-1}\Sigma_k^{xr}.$$

The responsibility for each component is:

$$h_k(x^*) = \frac{\pi_k \mathcal{N}(x^* \mid \mu_k^x, \Sigma_k^{xx})}{\sum_{j=1}^K \pi_j \mathcal{N}\big(x^* \mid \mu_j^x, \Sigma_j^{xx}\big)}.$$

The overall predicted residual and its variance are:

$$\hat{r}(x^*) = \sum_{k=1}^K h_k(x^*)\, \mu_k^{r|x},$$

$$\hat{\sigma}^2(x^*) = \sum_{k=1}^K h_k(x^*) \left[\sigma_k^{2|x} + \Big(\mu_k^{r|x} - \hat{r}(x^*)\Big)^2\right].$$

Thus, the final prediction is:

$$\hat{y}(x^*) = \hat{y}_{\text{GPNAM}}(x^*) + \hat{r}(x^*),$$

with predictive uncertainty quantified by:

$$\hat{\sigma}(x^*) = \sqrt{\hat{\sigma}^2(x^*)}.$$

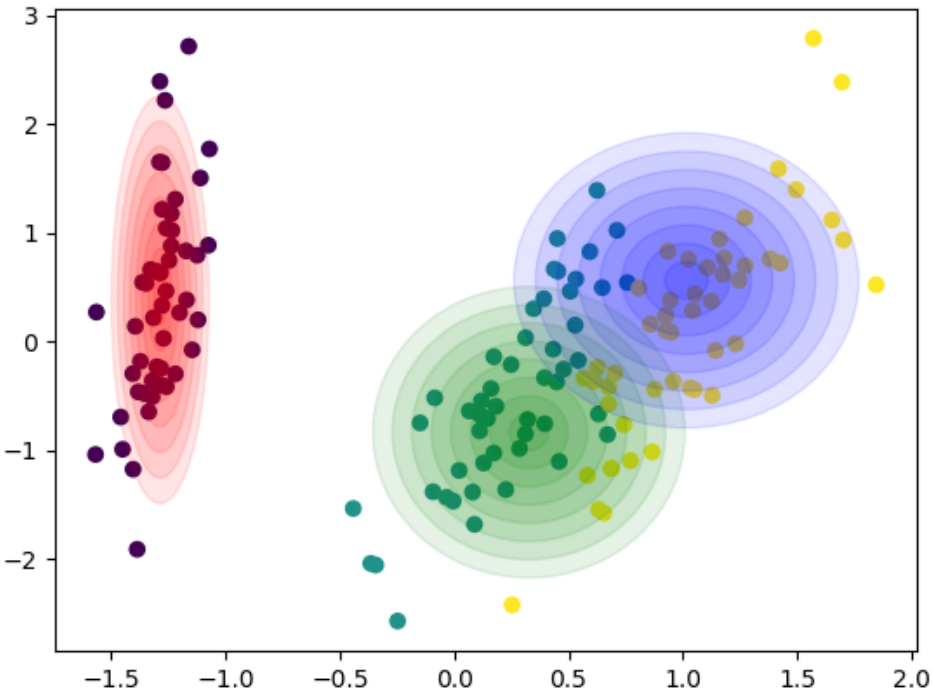

Figure 3.2: Representation of Gaussian Mixture Model

3.4 Framework with Neural Network-based Mixture Density Network

We begin with the point prediction from our GP-NAM model, denoted as $\hat{y}_{\text{base}}$. The key idea is to model the uncertainty and any remaining error by focusing on the residuals, defined as

$$r = y - \hat{y}_{\text{base}},$$

where y is the true target value. Instead of assuming these residuals are simply random noise, we use a Mixture Density Network (MDN)[29] to learn their probability distribution.

Modeling Residuals with an MDN Implemented as a Neural Network

The MDN is implemented as a neural network[30], which consists of the following key components [31]:

1. **Hidden Layer:** A non-linear activation (ReLU) is applied to capture complex patterns in the residuals.

2. **Output Layer:** The network outputs $3m$ values for a mixture of m Gaussian components. These outputs are split into three groups:

   - **Means:** $\mu_1, \mu_2, \ldots, \mu_m$.

   - **Log-Standard Deviations:** $\tilde{\sigma}_1, \tilde{\sigma}_2, \ldots, \tilde{\sigma}_m$, which are transformed to standard deviations as
   $$\sigma_i = \exp\big(\tilde{\sigma}_i\big),$$
   ensuring that $\sigma_i > 0$.

   - **Mixing Coefficients:** $\alpha_1^*, \alpha_2^*, \ldots, \alpha_m^*$, which are converted to valid mixing coefficients via the softmax function:
   $$\alpha_i = \frac{\exp\big(\alpha_i^*\big)}{\sum_{j=1}^{m} \exp\big(\alpha_j^*\big)},$$
   so that
   $$\sum_{i=1}^{m} \alpha_i = 1.$$

Using these parameters, the MDN models the conditional probability density of the residual as

$$p(r \mid x) = \sum_{i=1}^{m} \alpha_i \, \mathcal{N}\big(r; \mu_i, \sigma_i^2\big).$$

Expected Value and Variance of the MDN

The MDN provides a full probabilistic description of the residual. The expected value of the residual is computed as

$$\mathbb{E}[r \mid x] = \sum_{i=1}^{m} \alpha_i \, \mu_i.$$

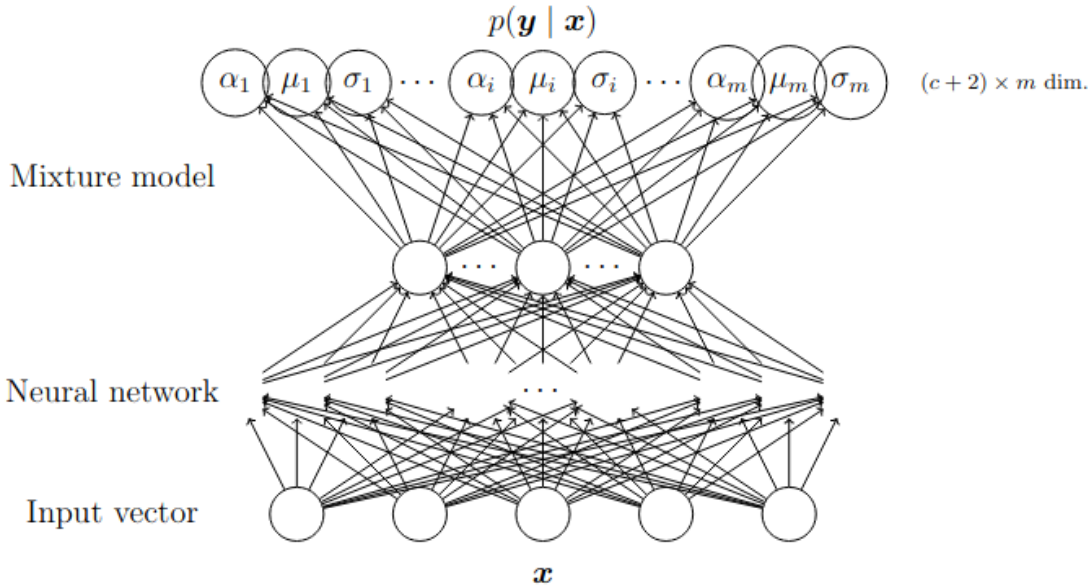

Figure 3.3: Representation of the Mixture Density Network model

In addition, the variance of the residual—accounting for both the uncertainty within each Gaussian component and the dispersion of the means—is computed as

$$\mathrm{Var}(r \mid x) = \sum_{i=1}^{m} \alpha_i \left(\sigma_i^2 + \mu_i^2\right) - \left(\sum_{i=1}^{m} \alpha_i \, \mu_i\right)^2 .$$

Final Prediction

The final prediction is obtained by combining the base prediction with the expected residual:

$$\hat{y} = \hat{y}_{\text{base}} + \mathbb{E}[r \mid x].$$

This formulation allows the model to not only adjust the point prediction from the GP-NAM model but also to quantify the uncertainty associated with the prediction.

Joint Optimization of the Whole Model

A central feature of this approach is the joint, end-to-end optimization of both

the base GP-NAM model and the MDN. The loss function used in training is a composite of two parts:

1. **Base Model Loss:** A mean squared error (MSE) loss, with an additional L_2 regularization term on the base model parameters, is used to ensure accurate point predictions:

$$L_{\text{base}} = \frac{1}{2}\,\mathbb{E}\Big[(y - \hat{y}_{\text{base}})^2\Big] + \frac{1}{2}\|w\|^2.$$

2. **MDN Loss:** A negative log-likelihood (NLL) loss is employed to encourage the MDN to assign high probability to the observed residuals:

$$L_{\text{MDN}} = -\frac{1}{N}\sum_{j=1}^{N}\log\left(\sum_{i=1}^{m}\alpha_i^{(j)}\,\mathcal{N}\Big(r^{(j)};\mu_i^{(j)},\sigma_i^{(j)2}\Big)\right),$$

   where the superscript (j) denotes the parameters corresponding to the j-th data point.

The total loss is then given by:

$$L_{\text{total}} = L_{\text{base}} + L_{\text{MDN}},$$

which ensures that the base model minimizes the point prediction error while the MDN accurately captures the uncertainty in the residuals.

Implementation Highlights

- **Residual Computation:** The residuals, $r = y - \hat{y}_{\text{base}}$, are computed using the output of the GP-NAM model.
- **Neural Network-Based MDN:** The MDN is implemented as a neural network that takes the input (or shared features) and outputs $3m$ parameters, which are then transformed (via exponentiation and softmax) to obtain the Gaussian mixture parameters.
- **Combined Prediction:** The final output is obtained by adding the expected value of the MDN-predicted residual, $\mathbb{E}[r \mid x]$, to the base prediction.

- **End-to-End Training:** Both components are optimized simultaneously using the composite loss L_{total}, allowing the model to learn complementary representations.
- **Adaptive Learning Rate:** A warm-up cosine decay schedule is employed to stabilize training, particularly during the early epochs.

Conclusion

This method leverages the strengths of deterministic and probabilistic modeling. By taking the base predictions from the GP-NAM model and then applying an MDN—implemented via a neural network—to the residuals, the approach yields final predictions that are both accurate and accompanied by robust uncertainty estimates.

We have also worked on proving the optimal number of GPs for our proposed model as well as on the interpretability of our proposed model, though both aspects exhibit some shortcomings. I will explain each in detail along with their respective issues.

3.5 Analysis of Gaussian Process Optimal Configurations

3.5.1 Objective and White Noise Criteria

We have done a detailed analysis of the residuals obtained from two different Gaussian Process (GP) frameworks: a 3-GP framework and an extended 4-GP framework on the autompg dataset. The primary goal of this analysis was to determine whether the residuals exhibit the characteristics of white noise, which include having a **zero mean, constant variance (homoscedasticity), no correlation with input features, and no autocorrelation**. The 4-GP framework involved applying an additional GP to the residuals of the third GP to model both the mean and variance, aiming to improve the residual quality Additionally, paired t-tests compare performance metrics such as RMSE, ECE, and NLPD.

3.5.2 Testing Procedures

Tests Conducted:

- **Zero Mean Test:** A t-test determines if the mean of the residuals is statistically zero[32]. This involves formulating a null hypothesis that the series' mean is zero against an alternative hypothesis that it is not. The process begins by calculating the sample mean of the series, followed by computing a test statistic, typically through a one-sample t-test, where the sample mean is divided by the standard error (derived from the sample standard deviation and sample size). This statistic is then compared to a t-distribution to obtain a p-value. If the p-value is below a chosen significance level, such as 0.05, the null hypothesis is rejected, indicating the mean is not zero; otherwise, the result supports a zero mean, consistent with white noise.

- **Breusch-Pagan Test:** Evaluates homoscedasticity (p-value < 0.05 indicates heteroscedasticity).[33]The test evaluates whether the variance of the series is constant over time, as non-constant variance (heteroskedasticity) would indicate the series is not white noise. The procedure involves fitting a regression model, typically regressing the squared residuals (or the series itself, if testing directly) against time or other explanatory variables, and then testing whether the variance depends on these variables. Under the null hypothesis, the variance is constant (homoskedastic), consistent with white noise, while the alternative suggests heteroskedasticity. The test statistic, calculated as a function of the regression's explained sum of squares, follows a chi-squared distribution. A p-value below a significance level (e.g., 0.05) leads to rejecting the null, indicating non-constant variance and ruling out white noise.

- **Pearson Correlation:** Checks whether residuals correlate with the input features[34].To perform this test, the Pearson correlation coefficient is calculated producing a value between -1 and 1 . Under the null hypothesis, the correlation is zero, consistent with white noise, while the alternative suggests significant correlation. The test statistic, derived from the correlation coefficient and sample size, is compared to a t-distribution to compute a p-value.

If the p-value exceeds a significance level (e.g., 0.05), the null hypothesis is not rejected.

- **Durbin-Watson Test:** A value near 2 indicates no autocorrelation.[35]The test statistic, ranging from 0 to 4, is calculated as the ratio of the sum of squared differences between consecutive residuals to the sum of squared residuals, with a value near 2 indicating no autocorrelation (consistent with white noise), values below 2 suggesting positive autocorrelation, and values above 2 indicating negative autocorrelation. Under the null hypothesis, there is no autocorrelation, supporting the white noise property, while the alternative indicates its presence. The statistic is compared to critical values from Durbin-Watson tables, specific to the sample size and number of predictors, to determine significance. A result close to 2 with a non-significant p-value (e.g., greater than 0.05) supports the absence of autocorrelation, aligning with white noise.

3.5.3 Results

3-GP Configuration

- **Zero Mean:**
  - For Training dataset p value comes out to be 0.9858 and for Testing dataset p value is 0.0968.Interpretation: Residuals are statistically indistinguishable from zero.
- **Homoscedasticity:**
  - For Training dataset p value equals 0.0002 and for Testing p value is 0.0271.Interpretation: The residuals exhibit heteroscedasticity, failing the constant variance criterion.
- **Feature Independence:** No significant correlations (all $p > 0.05$).
- **Autocorrelation:** Durbin-Watson values near 2 (Training: 2.0540; Testing: 2.0569).No Autocorrelation .

4-GP Configuration

- **Zero Mean:**
  - For Training dataset p value equals 0.986 and for Testing p value is 0.0976.Interpretation: Residuals maintain a zero mean similar to the 3-GP setup.
- **Homoscedasticity:**
  - For Training dataset p value equals 0.0002 and for Testing p value is 0.0272.Interpretation: Heteroscedasticity persists, indicating non-constant variance.
- **Feature Independence:** No significant correlations.
- **Autocorrelation:** Durbin-Watson values near 2 (Training: 2.055; Testing: 2.056).

.No Autocorrelation .

3.5.4 Limitations

Both configurations satisfy zero mean, feature independence, and no autocorrelation; however, neither achieves constant variance (heteroscedasticity persists). Moreover, the paired t-tests reveal no statistically significant advantage for the 4-GP configuration over the simpler 3-GP model RMSE: $p = 0.9534$,ECE: $p = 0.7234$,NLPD: $p = 0.3692$. Hence, the 3-GP setup is considered optimal in terms of balancing performance and complexity.

Drawback: The analysis is based solely on the auto-mpg dataset. Data characteristics can vary widely, so the observed optimality is empirical rather than mathematically proven.

3.5.5 Conclusion

In summary, our evaluations indicate that while both GP configurations yield residuals with a zero mean, independent features, and no autocorrelation, neither

obtains constant variance. With performance metrics being statistically equivalent, the 3-GP configuration is preferred due to its simpler architecture.However, the findings have a fundamental drawback—the results are dataset specific. This means that the observed optimality of the 3-GP setup is empirical and may not generalize to other datasets. Therefore, while our analysis provides a strong indication that the 3-GP setup is preferable under the tested conditions, it does not serve as a universally optimal proof.

3.6 Interpretability Measures and Their Drawbacks

3.6.1 Incorporation of Interpretability Measures

Interpretability is a critical aspect of machine learning models, especially in applications where trust, accountability, and transparency are essential. An interpretable model is transparent and understandable to humans, allowing us to comprehend their decision-making processes.[36] [37]

In this work, we incorporate interpretability measures into our model by evaluating three key dimensions: complexity, fidelity, and decomposability [38] [39]. These dimensions were synthesized into an aggregated metric, termed the **Interpretability Index**, which provides a holistic assessment of the model's interpretability. In doing so, we acknowledge that the specific formulation of these terms is one of many possible approaches and may not be universally accepted.

Complexity

The complexity of a model refers to its structural and computational simplicity, which directly impacts its interpretability[36]. We quantified complexity using the total number of trainable parameters in the model, including:

- **GPNAM Model Parameters:** Computed as:

$$\text{num_params_rff} = S \cdot \text{feature count} + 1,$$

where S is the number of Fourier features and *feature count* is the number of input features.

- **GP Model Parameters:** These include the parameters associated with the Gaussian Process models (kernel parameters and inducing points).

The Complexity Score is computed using the formula:

$$\text{ComplexityScore} = \frac{1}{1 + \lambda \cdot \text{total params}},$$

where λ is a regularization factor that controls the penalty for model size. A lower number of parameters leads to a higher Complexity Score, indicating a simpler and more interpretable model.

Fidelity

Fidelity measures how well a simpler surrogate model replicates the predictions of the complex model [36]. To evaluate fidelity:

- **Surrogate Model:** A Decision Tree Regressor, known for its inherent interpretability, is used as the surrogate model.
- **Training:** The Decision Tree is trained using the same input features as the complex model but with the complex model's predictions as the target variable.

The Fidelity Score is computed using the R^2 coefficient of determination between the surrogate model's predictions and the complex model's predictions:

$$\text{FidelityScore} = \max\Big(0, \min\Big(1, R^2\Big)\Big).$$

A high R^2 value indicates that the surrogate model closely approximates the behavior of the complex model, thereby enhancing interpretability.

Decomposability

Decomposability assesses whether the components of the model (e.g., features and parameters) are interpretable [38]. In our implementation, we penalize:

- The number of Gaussian Processes (num_gps),
- The dimensionality of the RFF space (rff dim).

The Decomposability Score is calculated as:

$$\text{DecomposabilityScore} = 1 - (\alpha_1 \cdot \text{num gps} + \alpha_2 \cdot \text{rff dim}),$$

where α_1 and α_2 are scaling factors that control the weight of each penalty term. A lower number of Gaussian Processes and lower RFF dimensions result in a higher Decomposability Score, reflecting improved interpretability.

3.6.2 Interpretability Index

The Interpretability Index aggregates the three dimensions — complexity, fidelity, and decomposability — into a single metric computed as:

$$\text{InterpretabilityIndex} = \alpha \cdot \text{ComplexityScore} + \beta \cdot \text{FidelityScore} + \gamma \cdot \text{DecomposabilityScore},$$

where α, β, and γ are weights that sum to 1.

3.6.3 Drawbacks and Limitations

While the framework provides a structured approach to assess interpretability, several drawbacks and limitations arise from the chosen formulation of the interpretability measures:

- **Subjectivity of Metric Definitions:**
  - The formulations for the Complexity, Fidelity, and Decomposability scores rely on specific, subjective choices. For example, counting the number of parameters as a measure of complexity or using a Decision Tree with a fixed maximum depth to assess fidelity are design decisions that may not capture all aspects of interpretability.
  - The weighting factors α, β, and γ in the Interpretability Index are arbitrarily chosen (in our case we have chosen to be 0.33,0.33,0.34) and

may require adjustment depending on the application.

- **Limited Scope of Metrics:**
  - The current formulation focuses on quantifiable metrics such as parameter counts and surrogate model performance. However, interpretability is a broader concept that also involves human factors, such as how easily non-expert stakeholders can understand the model.
- **Sensitivity to Hyperparameters:**
  - The interpretability metrics are sensitive to hyperparameters such as the regularization factor λ and the penalty coefficients α_1 and α_2. Small changes in these parameters can significantly alter the scores, which may lead to instability and reproducibility challenges.
- **Context-Dependence:**
  - Interpretability is highly context-dependent. What is considered interpretable in one domain might not be adequate in another, particularly in high-stakes fields like healthcare or finance where additional transparency and explanation criteria are crucial.

Recognizing these drawbacks is essential for refining the approach. Future work may involve incorporating additional dimensions or developing more flexible, context-aware metrics to construct a more universally acceptable interpretability framework.

3.6.4 Results

In repeated experiments on the Auto MPG dataset, we ran the model 10 times, shuffling the dataset before each train–test split. For each run, we computed the Interpretability Index as described above. When the base model used our proposed approach, the mean and standard deviation of the Interpretability Index across these 10 runs were:

$$\text{Interpretability Index} = 0.7228 \pm 0.0033.$$

We then replaced the base model with a neural network and repeated the same procedure of running 10 experiments on shuffled splits of the Auto MPG dataset. Under this alternative setup, the mean and standard deviation of the Interpretability Index were notably lower:

$$\text{Interpretability Index} = 0.4804 \pm 0.0051.$$

3.6.5 Conclusion

By incorporating these interpretability measures, we ensured that our model not only delivers accurate predictions but also remains comprehensible and transparent. This approach bridges the gap between black-box models and the need for interpretability in real-world applications.

Chapter 4

Results and Discussion

4.1 Evaluation Metrics

The model is evaluated using the following metrics:

- **Root Mean Square Error (RMSE):** Measures the closeness of predicted values to actual target values. Lower RMSE indicates better predictive accuracy.

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\Big(y_i - \hat{y}(x_i)\Big)^2},$$

- **Expected Calibration Error (ECE):** Assesses the alignment between model confidence and actual accuracy, with lower ECE values indicating good calibration.

- **Negative Log Predictive Density (NLPD):** Evaluates the quality of the model's uncertainty estimates. Lower NLPD implies a better representation of predictive uncertainty.

$$\text{NLPD} = -\frac{1}{N}\sum_{i=1}^{N}\log\left[\frac{1}{\sqrt{2\pi\,\hat{\sigma}^2(x_i)}}\exp\left(-\frac{\Big(y_i - \hat{y}(x_i)\Big)^2}{2\,\hat{\sigma}^2(x_i)}\right)\right],$$

Dataset

We consider the publicly available UCI regression datasets, where all the datasets are real-world regression problems.These datasets offer a diverse range of complexities and sizes, making them ideal for thoroughly evaluating the capabilities of the proposed hybrid model.

4.2 Results

The model was tested on several datasets. The table below summarizes the results, displaying RMSE, ECE, and NLPD values (mean ± standard deviation) for each dataset on testing data .All the experiments are run for 10 times by shuffling the dataset before train-test split each time and the mean and standard deviation of the metrics are reported.

Table 4.1: Performance comparison (RMSE ↓, ECE ↓, and NLPD ↓) across multiple datasets. Minimum values for each metric are highlighted in bold.

	Auto MPG	Machine	Energy	Concrete	Wine	Yacht
			RMSE ↓			
Framework with GMM	0.0861 ± 0.0134	**0.0278 ± 0.0184**	**0.0467 ± 0.0175**	0.1052 ± 0.0080	0.1531 ± 0.0213	0.1538 ± 0.0224
Proposed Model	**0.0768 ± 0.0086**	0.0523 ± 0.0337	0.0698 ± 0.0193	**0.0959 ± 0.0044**	**0.1369 ± 0.0147**	**0.0674 ± 0.0175**
Framework with MDN	0.1391 ± 0.0109	0.1135 ± 0.0454	0.1994 ± 0.0367	0.2246 ± 0.0254	0.2033 ± 0.0319	0.2387 ± 0.0237
			ECE ↓			
Framework with GMM	0.0206 ± 0.0070	**0.0078 ± 0.0038**	**0.0123 ± 0.0053**	0.0201 ± 0.0047	0.0655 ± 0.0169	0.0504 ± 0.0145
Proposed Model	**0.0161 ± 0.0050**	0.0180 ± 0.0120	0.0329 ± 0.0191	**0.0200 ± 0.0042**	**0.0470 ± 0.0106**	**0.0345 ± 0.0103**
Framework with MDN	0.0774 ± 0.0173	0.0390 ± 0.0289	0.1150 ± 0.0339	0.0997 ± 0.0365	0.0953 ± 0.0417	0.0852 ± 0.0575
			NLPD ↓			
Framework with GMM	−17.2591 ± 9.4235	−12.2877 ± 11.0701	−47.2010 ± 19.7770	−4.8749 ± 1.2697	−3.7984 ± 1.2073	−20.8827 ± 7.8678
Proposed Model	1.1653 ± 0.0565	1.0959 ± 0.7599	0.9524 ± 0.1385	0.5002 ± 0.0516	0.5617 ± 0.1128	0.6292 ± 0.1059
Framework with MDN	−1.9523 ± 0.0502	−1.9142 ± 0.0392	−1.9519 ± 0.1111	−1.9800 ± 0.1336	−2.0689 ± 0.1558	−1.9839 ± 0.0907

We have also compared our results with other state of arts predictive distribution models including Bayesian neural network [40] , Bayesian Linear Regression[41],Quantile regression[42],Natural Gradient Boosting[43] and Gaussian Process Regression[4] .

Bayesian neural network

Bayesian neural networks extend conventional neural networks by treating each weight not as a fixed scalar but as a probability distribution, thus capturing uncertainty in model parameters and predictions. By placing priors over weights and

computing a posterior given data, BNNs naturally quantify epistemic uncertainty while mitigating overfitting through Bayesian regularization.

Exact Bayesian inference in BNNs is intractable for modern architectures due to the high-dimensional integrals over weight space. Approximate methods such as Markov chain Monte Carlo[12] and variational inference techniques[44] are used to approximate the posterior distribution on weights.

Bayesian Linear Regression

Bayesian linear regression treats each model coefficient as a probability distribution rather than a fixed value, so learning combines prior beliefs with data to update how confident we are in each coefficient. Predictions then come with built-in "error bars" that grow when data are limited, naturally regularizing the model and quantifying uncertainty[41] [45] .

Quantile regression

Quantile regression extends traditional least-squares approaches by estimating specified percentiles of the outcome's conditional distribution instead of focusing solely on its mean. Unlike mean regression, quantile regression fits each quantile by minimizing an asymmetric check-function loss that penalizes over- and under-prediction differently, yielding regression lines that hug the chosen quantile of the data. The result is a set of conditional quantile curves that together describe the full response distribution[42].

Natural Gradient Boosting

Natural Gradient Boosting is a probabilistic extension of gradient boosting that models full conditional distributions rather than just point estimates by treating the parameters of a chosen probability family as boosting targets and updating them via the natural gradient. This yields well-calibrated uncertainty estimates alongside accurate predictions [43].

Dataset	RMSE	NLPD	ECE
Autompg	2.9589 ± 0.2957	-3.2866 ± 0.4836	0.0177 ± 0.0039
Machine	18.6129 ± 8.1490	-3.3916 ± 0.6269	0.0151 ± 0.0027
Energy	2.0828 ± 0.3438	-2.1284 ± 0.2711	0.0070 ± 0.0015
Concrete	5.8321 ± 0.3259	-4.9266 ± 0.3202	0.0093 ± 0.0009
Wine	1.0435 ± 0.1720	-1.4694 ± 0.2452	0.0118 ± 0.0022
Yacht	1.2815 ± 0.3634	-0.8097 ± 0.1803	0.0087 ± 0.0023

Table 4.2: Evaluation results of Bayesian Neural Network on different datasets

Dataset	RMSE	NLPD	ECE
Autompg	3.5177 ± 0.3260	-2.6840 ± 0.0904	0.0095 ± 0.0015
Machine	35.6586 ± 11.0597	-5.0160 ± 0.3783	0.0117 ± 0.0029
Energy	3.0291 ± 0.1229	-2.5307 ± 0.0438	0.0094 ± 0.0011
Concrete	10.5886 ± 0.6957	-3.7824 ± 0.0684	0.0060 ± 0.0016
Wine	1.0435 ± 0.1720	-1.4694 ± 0.2452	0.0118 ± 0.0022
Yacht	8.9924 ± 0.6229	-3.6189 ± 0.0675	0.0097 ± 0.0023

Table 4.3: Evaluation results of Bayesian Linear Regression on different datasets

Dataset	RMSE	NLPD	ECE
Autompg	3.7430 ± 0.2990	-4.3544 ± 1.6703	0.0142 ± 0.0024
Concrete	8.4664 ± 0.6011	-3.7676 ± 0.1609	0.0058 ± 0.0009
Energy	3.6802 ± 0.2034	-2.9977 ± 0.5896	0.0068 ± 0.0016
Machine	84.982 ± 47.138	-163.482 ± 298.952	0.0150 ± 0.0040
Yacht	5.4437 ± 0.8101	-3.4744 ± 0.2612	0.0243 ± 0.0039
Wine	0.5336 ± 0.0556	-5.0315 ± 1.8206	0.0269 ± 0.0045

Table 4.4: Evaluation results of quantile-regression on different datasets.

Dataset	RMSE	NLPD	ECE
Autompg	2.8140 ± 0.3737	-3.2918 ± 0.8621	0.0172 ± 0.0028
Concrete	5.4960 ± 0.2720	-3.0890 ± 0.0634	0.0057 ± 0.0014
Energy	0.5064 ± 0.0286	-0.6022 ± 0.1138	0.0063 ± 0.0012
Machine	36.5306 ± 17.1509	-49.6446 ± 37.4962	0.0180 ± 0.0025
Yacht	0.7217 ± 0.2169	-0.8477 ± 0.6324	0.0126 ± 0.0035
Wine	0.4954 ± 0.0686	-3.1769 ± 0.9302	0.0276 ± 0.0034

Table 4.5: Evaluation results of Natural Gradient Boosting on different datasets.

Dataset	RMSE	NLPD	ECE
Autompg	2.6761 ± 0.3641	-2.4117 ± 0.1324	0.7985 ± 0.1951
Concrete	5.7201 ± 0.3414	-3.1083 ± 0.0491	1.0444 ± 0.1908
Energy	0.4979 ± 0.0499	-0.6722 ± 0.1047	0.1070 ± 0.0198
Machine	137.7400 ± 43.7287	-6.5439 ± 0.5183	50.3100 ± 8.4077
Yacht	0.9405 ± 0.9413	-0.2910 ± 0.1579	0.2492 ± 0.2382
Wine	0.5617 ± 0.0574	-0.8506 ± 0.0979	0.2291 ± 0.0428

Table 4.6: Evaluation results of Gaussian Process Regressor on different datasets.

4.3 Results Comparison

In these experiments, our proposed model with Gaussian Process consistently delivered low predictive errors (RMSE) while exhibiting balanced uncertainty estimates across most of the evaluated datasets. Their performance was especially notable in Auto MPG, Concrete, Wine, and Yacht, where they achieved the lowest RMSE and demonstrated relatively low calibration errors (ECE). In comparison, Framework with Gaussian Mixture Models at times outperformed Gaussian Processes in terms of raw error (as observed in the Machine and Energy datasets), but they did so by assigning very small variance to predictions, leading to extremely negative NLPD values. While a lower NLPD is generally desirable, in this case it signals that GMMs are overconfident rather than genuinely superior. Lastly, Mixture Density Networks tended to lag behind both GMMs and GPs, showing higher RMSE scores and less favorable calibration metrics. Overall, Gaussian Processes emerged as the most balanced method, offering both strong predictive accuracy and more reliable uncertainty quantification.

Our proposed model is performing far better in terms of RMSE with respect to Bayesian Neural Network, Bayesian Linear Regression, Quantile Regression, Natural Gradient Boosting and Gaussian Process Regressor for all datasets but in terms of NLPD it doesnot perform well with respect to these model .

4.4 Conclusion

This study presents a novel approach to uncertainty estimation by developing a hybrid model that integrates Random Fourier Features (RFF), Gaussian Processes (GPs), and Neural Additive Models (NAMs). By combining these methodologies, we achieve a model that is not only accurate and computationally viable but also offers insights into both the predictive mechanisms and the associated uncertainties.

The model was evaluated across multiple datasets, demonstrating strong predictive accuracy and well-calibrated uncertainty estimates. This model's performance suggests that it is a viable tool for practical applications where interpretability, reliability, and computational efficiency are required.

4.5 Future Works

An exciting direction for future research is to extend our proposed model to classification tasks.

Chapter 5

Appendix

5.1 Variational Sparse Gaussian Process Approximation

Gaussian Processes (GPs) provide a flexible Bayesian framework for regression and classification, modeling functions via a mean function $m(x)$ and a covariance kernel $k(x, x')$. However, standard GPs scale poorly, with $O(n^3)$ complexity for n data points due to covariance matrix inversion. The variational sparse GP approximation, introduced by [25], addresses this by using $m \ll n$ inducing points, reducing complexity to $O(nm^2 + m^3)$.

Overview

Given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, a GP prior is placed over latent function values $f = [f(x_1), \ldots, f(x_n)]^T$, with $p(f) = \mathcal{N}(0, K_{XX})$, where $K_{XX} = [k(x_i, x_j)]$. For regression, observations follow $y_i = f(x_i) + \epsilon_i$, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$. Inducing points $Z = \{z_1, \ldots, z_m\}$ and their latent values $u = [f(z_1), \ldots, f(z_m)]^T$ are introduced, with prior $p(u) = \mathcal{N}(0, K_{ZZ})$. The joint prior is:

$$p(f, u) = p(f|u)p(u),$$

where $p(f|u) = \mathcal{N}(K_{XZ}K_{ZZ}^{-1}u, K_{XX} - K_{XZ}K_{ZZ}^{-1}K_{ZX})$, and $K_{XZ} = [k(x_i, z_j)]$.

Variational Inference

The posterior $p(f, u|y)$ is intractable, especially for non-Gaussian likelihoods. A variational distribution $q(f, u) = p(f|u)q(u)$, with $q(u) = \mathcal{N}(m, S)$, approximates it. The Evidence Lower Bound (ELBO) is maximized to optimize variational parameters m, S, inducing points Z, and kernel/noise hyperparameters θ:

$$\mathcal{L} = \mathbb{E}_{q(f)}[\log p(y|f)] - \mathrm{KL}(q(u)||p(u)).$$

For regression, the ELBO is:

$$\mathcal{L} = \sum_{i=1}^{n} \left[-\frac{1}{2\sigma^2} \mathbb{E}_{q(f_i)}[(y_i - f_i)^2] - \frac{1}{2} \log(2\pi\sigma^2) \right] - \mathrm{KL}(\mathcal{N}(m, S)||\mathcal{N}(0, K_{ZZ})).$$

The expectation is computed analytically, as $q(f)$ is Gaussian. For classification, numerical methods (e.g., quadrature) handle non-Gaussian likelihoods.

Optimization and Prediction

The ELBO is optimized using gradient-based methods (e.g., Adam), treating Z as variational parameters. For a test point x^*, the predictive distribution is:

$$q(f^*) = \mathcal{N}(\mu^*, \Sigma^*),$$

where $\mu^* = K_{x^*Z} K_{ZZ}^{-1} m$, and $\Sigma^* = K_{x^*x^*} - K_{x^*Z} K_{ZZ}^{-1} K_{Zx^*} + K_{x^*Z} K_{ZZ}^{-1} S K_{ZZ}^{-1} K_{Zx^*}$.

Bibliography

[1] Samuel S Blackman. Multiple hypothesis tracking for multiple target tracking. *IEEE Aerospace and Electronic Systems Magazine*, 19(1):5–18, 2004.

[2] Laurens Sluijterman, Eric Cator, and Tom Heskes. How to evaluate uncertainty estimates in machine learning for regression? *Neural Networks*, 173:106203, 2024.

[3] Ali Rahimi and Benjamin Recht. Random features for large-scale kernel machines. In *Advances in Neural Information Processing Systems*, page 1177–1184, 2007.

[4] Christopher KI Williams and Carl Edward Rasmussen. *Gaussian processes for machine learning*, volume 2. MIT press Cambridge, MA, 2006.

[5] Rishabh Agarwal, Levi Melnick, Nicholas Frosst, Xuezhou Zhang, Ben Lengerich, Rich Caruana, and Geoffrey E Hinton. Neural additive models: Interpretable machine learning with neural nets. *Advances in neural information processing systems*, 34:4699–4711, 2021.

[6] Eyke Hüllermeier and Willem Waegeman. Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods. *Machine learning*, 110(3):457–506, 2021.

[7] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[8] Edward Snelson and Zoubin Ghahramani. Sparse gaussian processes using pseudo-inputs. In *Advances in Neural Information Processing Systems*, page 1257–1264, 2006.

[9] Wei Zhang, Brian Barr, and John Paisley. Gaussian process neural additive models. In *arXiv preprint arXiv:2402.12518*, 2024.

[10] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. *International Conference on Machine Learning*, page 1050–1059, 2016.

[11] Charles Blundell and et al. Weight uncertainty in neural networks. In *International Conference on Machine Learning*, page 1613–1622, 2015.

[12] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*, pages 1050–1059. PMLR, 2016.

[13] Balaji Lakshminarayanan and et al. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, page 6405–6416, 2017.

[14] Andreas Beyerlein. Quantile regression—opportunities and challenges from a user's perspective. *American journal of epidemiology*, 180(3):330–331, 2014.

[15] Matteo Fontana, Gianluca Zeni, and Simone Vantini. Conformal prediction: a unified review of theory and new challenges. *Bernoulli*, 29(1):1–23, 2023.

[16] Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? *Advances in neural information processing systems*, 30, 2017.

[17] Hristos Tyralis and Georgia Papacharalampous. A review of predictive uncertainty estimation with machine learning. *Artificial Intelligence Review*, 57(4):94, 2024.

[18] Haitao Liu, Yew-Soon Ong, Xiaobo Shen, and Jianfei Cai. When gaussian process meets big data: A review of scalable gps. *IEEE transactions on neural networks and learning systems*, 31(11):4405–4423, 2020.

[19] Dan Levi, Liran Gispan, Niv Giladi, and Ethan Fetaya. Evaluating and calibrating uncertainty prediction in regression tasks. *Sensors*, 22(15):5540, 2022.

[20] Moloud Abdar, Farhad Pourpanah, Sadiq Hussain, Dana Rezazadegan, Li Liu, Mohammad Ghavamzadeh, Paul Fieguth, Xiaochun Cao, Abbas Khosravi, U Rajendra Acharya, et al. A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information fusion*, 76:243–297, 2021.

[21] Paul W. Goldberg, Christopher K. I. Williams, and Charles M. Bishop. Regression with input-dependent noise: A gaussian process treatment. In *Neural Information Processing Systems*, 1997.

[22] Miguel Lázaro-Gredilla and Michalis K Titsias. Variational heteroscedastic gaussian process regression. In *ICML*, pages 841–848, 2011.

[23] Jarno Vanhatalo, Jaakko Riihimäki, Jouni Hartikainen, Pasi Jylänki, Ville Tolvanen, and Aki Vehtari. Bayesian modeling with gaussian processes using the gpstuff toolbox. *arXiv preprint arXiv:1206.5754*, 2012.

[24] Daniel Hernández-Lobato, Viktoriia Sharmanska, Kristian Kersting, Christoph H Lampert, and Novi Quadrianto. Mind the nuisance: Gaussian process classification using privileged noise. *Advances in Neural Information Processing Systems*, 27, 2014.

[25] James Hensman, Alexander Matthews, and Zoubin Ghahramani. Scalable variational gaussian process classification. In *Artificial intelligence and statistics*, pages 351–360. PMLR, 2015.

[26] Douglas A Reynolds et al. Gaussian mixture models. *Encyclopedia of biometrics*, 741(659-663):3, 2009.

[27] Alexander Fabisch. gmr: Gaussian mixture regression. *Journal of Open Source Software*, 6(62):3054, 2021.

[28] Zoubin Ghahramani and Michael Jordan. Supervised learning from incomplete data via an em approach. *Advances in neural information processing systems*, 6, 1993.

[29] Christopher M Bishop. Mixture density networks. 1994.

[30] Axel Brando. Mixture density networks (mdn) for distribution and uncertainty estimation, 2017. Report of the Master's Thesis: Mixture Density Networks for distribution and uncertainty estimation.

[31] Axel Brando. Mixture density networks (mdn) for distribution and uncertainty estimation, 2017. GitHub repository with a collection of Jupyter notebooks intended to solve a lot of problems related to MDN.

[32] Cécile Durot and Yves Rozenholc. An adaptive test for zero mean. *Mathematical Methods of Statistics*, 15(1):26–60, 2006.

[33] Andreea G Halunga, Chris D Orme, and Takashi Yamagata. A heteroskedasticity robust breusch–pagan test for contemporaneous correlation in dynamic panel data models. *Journal of econometrics*, 198(2):209–230, 2017.

[34] Israel Cohen, Yiteng Huang, Jingdong Chen, Jacob Benesty, Jacob Benesty, Jingdong Chen, Yiteng Huang, and Israel Cohen. Pearson correlation coefficient. *Noise reduction in speech processing*, pages 1–4, 2009.

[35] Kenneth J White. The durbin-watson test for autocorrelation in nonlinear models. *The Review of Economics and Statistics*, pages 370–373, 1992.

[36] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature machine intelligence*, 1(5):206–215, 2019.

[37] Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.

[38] Zachary C Lipton. The mythos of model interpretability: In machine learning, the concept of interpretability is both important and slippery. *Queue*, 16(3):31–57, 2018.

[39] W James Murdoch, Chandan Singh, Karl Kumbier, Reza Abbasi-Asl, and Bin Yu. Definitions, methods, and applications in interpretable machine learning. *Proceedings of the National Academy of Sciences*, 116(44):22071–22080, 2019.

[40] Ethan Goan and Clinton Fookes. Bayesian neural networks: An introduction and survey. *Case Studies in Applied Bayesian Data Science: CIRM Jean-Morlet Chair, Fall 2018*, pages 45–87, 2020.

[41] Thomas Minka. Bayesian linear regression. Technical report, Citeseer, 2000.

[42] Roger Koenker and Kevin F Hallock. Quantile regression. *Journal of economic perspectives*, 15(4):143–156, 2001.

[43] Tony Duan, Avati Anand, Daisy Yi Ding, Khanh K Thai, Sanjay Basu, Andrew Ng, and Alejandro Schuler. Ngboost: Natural gradient boosting for probabilistic prediction. In *International conference on machine learning*, pages 2690–2700. PMLR, 2020.

[44] Alex Graves. Practical variational inference for neural networks. *Advances in neural information processing systems*, 24, 2011.

[45] Scott A Baldwin and Michael J Larson. An introduction to using bayesian linear regression with clinical data. *Behaviour research and therapy*, 98:58–75, 2017.